NEHAL ALI KHAN 25K-4133 BAI-3A Date: M T W T F S S

e1)
```
mov ax, 3
mov bx, ax
shl ax, 4
mov cx, bx
shl cx, 3
add ax, a
shl bx, 1
add ax, bx
```

ex)
```
.data
Val DWORD 2345ABCDh ;

.code
mov ecx, 4
R1: mov al, BYTE PTR Val
shr al, 1
rcr BYTE PTR Val+3, 1
rcr BYTE PTR Val+2, 1
rcr BYTE PTR Val+1, 1
rcr BYTE PTR Val+0, 1
loop R1
; Val = 02345ABCh

mov Val, 2345ABCDh
mov ecx, 4
L1: mov al, BYTE PTR Val+3
shl al, 1
rcl BYTE PTR Val, 1
rcl BYTE PTR Val+1, 1
rcl BYTE PTR Val+2, 1
rcl BYTE PTR Val+3, 1
loop L1
; Val=345ABCD2h
```

Date:
M T W T F S S

Start 2345ABCDh

1 91A2D5E6h

2 48D16AF3h

3 A468B579h

4 D2345ABCh

Q3)

```
        mov ecx , 2
L2:     mov ax , WORD PTR Val
        shr ax , 1
        rcr WORD PTR Val+2 , 1
        rcr WORD PTR Val , 1
        loop L2
; Val = 48D16AF3h
```

```
        mov Val, 2345ABCDh
        mov ecx, 2
L2:     mov ax, WORD PTR Val+2
        shl ax, 1
        rcl WORD PTR Val, 1
        rcl WORD PTR Val+2, 1
        loop L2
; Val = 8D16AF34h
```

Op	Pass 1	Pass 2
Right	91A2D5E6h	48D16AF3h
Left	46B579Ah	8D16AF34h

Date:
M T W T F S S

Q4) Dx = 0101100111011010

Hours = 01011 = 11

Minutes = 001110 = 14

Seconds = 11010 = 26

a) mov dx, 0101100111011010b
mov al, dl
and al, 00011111b
mov BSeconds, al

b) mov ax, dx
shr ax, 5
and al, 00111111b
mov BMinutes, al

Q5)

			AX	DX	CF
-	mov dx, 9A3Fh | mov ax, 7FE0h | clc		7FE0	9A3F	0
1	add ax, 90EAh	7FE0+90EA = 106A	106A	9A3F	1
2	adc dx, 1	9A3F +1 +CF(1) = 9A41	106A	9A41	0
→	stc		106A	9A41	1
3	add ax, E120h	106A+E120 = 91EA	91EA	9A41	0
4	adc dx, 0FFFFh	9A41+FFFF = 19A40	91EA	9A40	1

Q6) a) ZF = 1 if AX is zero

test ax, 0FFFFh (result is 0 only when AX = 0)

b) ZF cleared if BX is odd

test bx, 1 (odd means bit = 0 = 1, so result ≠ 0 and ZF = 0)

c) SF = 1 if DX is negative

test dx, 8000h (negative means bit 15 = 1, so result = 8000h and SF = 1)

Date:
M T W T F S S

d) ZF = 1 if DX is zero or positive

test dx, 8000h (bit 15 = 0 makes result 0, so ZF = 1)

e) PF = 1 if BL has an even number of 1s

test bl, 0FFh (or test bl, bl) PF = 1 when the low byte of the result has even parity)

f)
```
mov dx, 5
stc
mov ax, 10h
sbc dx, ax
```

g)
```
.data
val1 QWORD 2040300436204 7A1h
val2 QWORD 0552103041A2630B2h
result QWORD 0

.code
main PROC
    mov ecx, 8
    mov esi, OFFSET val1
    mov edi, OFFSET val2
    mov ebx, OFFSET result

lup:
    mov al, BYTE PTR [esi]
    sbb al, BYTE PTR [edi]
    mov BYTE PTR [ebx], al
    inc esi
    inc edi
    inc ebx
    loop lup
```

Date:
M T W T F S S

exit

main ENDP

Q9) INCLUDE Irvine32.inc

.data

prompt BYTE "Enter marks (0-100): ", 0

msgA BYTE "Grade: A", 0

msgB BYTE "Grade : B ", 0

msg C BYTE "Grade : C ", 0

msg F BYTE "Grade : F", 0

msgBad BYTE "Invalid marks! ", 0

.code

main PROC

mov edx, OFFSET prompt

call WriteString

call ReadInt

cmp eax, 0

jl Invalid

cmp eax, 100

jg Invalid

cmp ax, 80

jge GradeA

cmp ax, 60

jge Grade B

cmp eax, 40

jge Grade C

mov edx, OFFSET msgF

jmp show

Date:
M T W T F S S

```
Grade A :   mov edx , OFFSET msgA
            jmp show
Grade B :  mov edx , OFFSET msg B
            jmp show
Grade C :  mov edx, OFFSET msg C
            jmp show
Invalid :  mov edx , OFFSET msgBad
show :
            call WriteString
            call crlf
            exit
main ENDP
END main
```

(iv) 1) empty
2) empty
3) 9
4) 3.7
5) 3.8
6) 9, 10
7) 8
8) empty

(iii)
```
.data
num1   SDWORD 0
num 2  SDWORD ?
num 3  SDWORD ?
num 4  SDWORD ?
result SDWORD ?
```

Date:
M T W T F S S

```
.code
    mov  eax, num1
    cmp  eax, num2
    jle  L-elseit
    mov  eax, num3
    cmp  eax, num4
    je   L-elseit
    mov  eax, num1
    add  eax, num3
    mov  result, eax
    jmp  L_end

L_elseit :
    mov  eax, num2
    cmp  eax, num4
    jne L_else
    mov  eax, num2
    sub  eax, num1
    mov result, eax
    jmp  L-end
L_else :
    mov result, 0

L_end :
```

(Q12)

```
; data
    op1   SDWORD ?
    op2   SDWORD ?
    x     SDWORD ?
    y     SDWORD ?
    z     SDWORD ?

. code
    mov eax, op1
```

SW
SHAHEER PAPER PRODUCTS

Date:
M T W T F S S

```
L-while :
    cmp  eax, up2
    jg   L-endwhile

cmp eax, x
 jl  L-else
cmp  eax, y
jg  L-else
odd  2, 10
jmp L-next
```